

Python Programming

Detailed Chapter Notes & Worked Examples - Volume 2

This pack develops practical understanding through explanations, worked examples, lab tasks and review questions. Complete the matching Foundation Pack first.

Chapter	Topic
1	Variables and decisions
2	Loops and collections
3	Functions and files
4	Program project

Chapter 1: Variables and decisions

Concept notes

Choose meaningful names and correct data types. Validate input before calculation. Conditions should cover boundary and invalid cases.

Worked example

`marks=float(input()); if 0 <= marks <= 100 classify result, otherwise display invalid input.`

Lab assignment

Build grade calculator with tests for -1, 0, 39, 40, 100 and 101.

Review questions

1. Why test boundaries?
2. Difference int and float?
3. What makes a name meaningful?

Evidence to submit: source file or working output, screenshot/PDF, and a 100-word explanation of decisions and checks.

Chapter 2: Loops and collections

Concept notes

Lists preserve ordered items; dictionaries map keys to values. Loops repeat processing; accumulators build totals.

Worked example

Store students as dictionaries, loop through records and calculate class average and highest mark.

Lab assignment

Create a five-student report without repeating print code.

Review questions

1. When use dictionary?
2. What is accumulator?
3. How avoid infinite loop?

Evidence to submit: source file or working output, screenshot/PDF, and a 100-word explanation of decisions and checks.

Chapter 3: Functions and files

Concept notes

A function performs one clear job with inputs and return value. File handling should specify encoding and use context manager.

Worked example

`read_students(path)` returns validated rows; `calculate_summary(rows)` returns totals; `save_report` writes output.

Lab assignment

Build CSV attendance summary using three functions and exception handling.

Review questions

1. Why return values?
2. Purpose of with open?
3. What should exception message explain?

Evidence to submit: source file or working output, screenshot/PDF, and a 100-word explanation of decisions and checks.

Chapter 4: Program project

Concept notes

Separate data, logic and interface. Test small functions, handle invalid input and keep backups or version history.

Worked example

Student manager supports add, search, list and save; IDs remain unique after restart.

Lab assignment

Build project, test 10 cases and prepare README.

Review questions

1. Why separate concerns?
2. How maintain unique IDs?
3. What belongs in README?

Evidence to submit: source file or working output, screenshot/PDF, and a 100-word explanation of decisions and checks.

Final Applied Assignment

Create one original project that combines all four chapters of Python Programming. Use realistic but non-sensitive sample data. Submit planning notes, source files, final output, test evidence and a one-page reflection.

Assessment rubric

Area	Marks
Concept accuracy	25
Working practical output	30
Validation and testing	20
File organisation and presentation	15
Original explanation	10

Student declaration

I confirm that the submitted work is my own practice. I have not included passwords, OTPs, identity documents or confidential student records.

Support: mcieducationalgroup@gmail.com | 9334779133, 7004773247